Figure 11.12 Continuing the example in figure 11.11 about running a partial query on a k-d tree. The third step, in the top part, shows another filtering step: nodes `D` and `E`'s split-coordinate is again x, so we can prune the branches on which we will search. On the last step, since we are at the leaf level, we only check if nodes `G` and F should be added to the results (only `F` will).

K-d trees provide a natural extension of that use case when we need to index tables with composite keys: while traversing the tree we will cycle through all the fields in the composite key. Moreover, k-d trees provide methods for exact match, best match (the nearest neighbor search) and range search; partial match queries could also be supported.

Although we haven't included partial queries in our containers' API, implementing them is straightforward – we perform a regular exact match query on the fields in the query, while we follow both branches in levels corresponding to fields of the keys that are not filtered in the query.

For instance, if we use a k-d tree for geospatial data, and we look for all the point on a line parallel to the x axis, such that `x==C`, we would traverse both branches in the nodes at odd levels (where we split on y), while at even levels (corresponding to x coordinate splits) we would only traverse the branch containing `C`. Figures 11.11 and 11.12 show how a partial range search works on a k-d tree.

Alternatively, we can also use the existing `pointsInRectangle` method, passing a range spanning from the minimum possible to the maximum possible value for those fields we are not going to put